

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Advanced Todo App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <h1>🌟 Todo List App</h1>

  <div class="top-bar">
    <span id="dateTime"></span>
    <button onclick="toggleTheme()">🌙</button>
  </div>

  <input type="text" id="taskInput" placeholder="Enter task...">

  <select id="category">
    <option>School</option>
    <option>Personal</option>
    <option>Work</option>
  </select>

  <input type="datetime-local" id="dueDate">

  <button onclick="addTask()">Add Task</button>

  <input type="text" id="search" placeholder="Search..."
onkeyup="renderTasks()">

  <!-- Progress -->
  <div class="progress-container">
    <div id="progressBar"></div>
  </div>
  <p id="progressText"></p>

  <div id="taskList"></div>
</div>

<script src="script.js"></script>
```

```
</body>
</html>
```

Style.css

```
:root {
  --bg: #f4f6fb;
  --card: #ffffff;
  --text: #333;
  --primary: #4f46e5;
}

body.dark {
  --bg: #0f172a;
  --card: #1e293b;
  --text: #e2e8f0;
}

body {
  margin: 0;
  font-family: "Segoe UI", sans-serif;
  background: var(--bg);
  color: var(--text);
  display: flex;
  justify-content: center;
  padding-top: 30px;
}

.container {
  width: 400px;
  background: var(--card);
  padding: 20px;
  border-radius: 16px;
  box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

h1 {
  text-align: center;
}

.top-bar {
  display: flex;
  justify-content: space-between;
```

```
margin-bottom: 10px;
}

input, select {
  width: 100%;
  padding: 10px;
  margin-bottom: 8px;
  border-radius: 8px;
  border: 1px solid #ccc;
}

button {
  padding: 10px;
  width: 100%;
  border: none;
  background: var(--primary);
  color: white;
  border-radius: 8px;
  cursor: pointer;
}

.task {
  padding: 10px;
  border-radius: 10px;
  margin-top: 10px;
  background: rgba(0,0,0,0.05);
  display: flex;
  justify-content: space-between;
  align-items: center;
  cursor: grab;
}

.task.completed {
  text-decoration: line-through;
  opacity: 0.6;
}

.task.overdue {
  border-left: 5px solid red;
}

.actions button {
  margin-left: 5px;
}
```

```
padding: 5px 8px;
}

.progress-container {
width: 100%;
height: 10px;
background: #ddd;
border-radius: 5px;
margin-top: 10px;
}

#progressBar {
height: 100%;
width: 0%;
background: green;
border-radius: 5px;
}

.category {
font-size: 12px;
}

.due {
font-size: 11px;
color: crimson;
}
```

[script.js](#)

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function save() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}

// 🕒 Time
setInterval(() => {
  document.getElementById("dateTime").innerText =
    new Date().toLocaleString();
}, 1000);

// 🌙 Theme
function toggleTheme() {
  document.body.classList.toggle("dark");
}
```

```

}

// + Add Task
function addTask() {
  const text = document.getElementById("taskInput").value;
  const category = document.getElementById("category").value;
  const dueDate = document.getElementById("dueDate").value;

  if (!text) return alert("Enter a task!");

  tasks.push({
    text,
    category,
    dueDate,
    completed: false
  });

  document.getElementById("taskInput").value = "";
  save();
  renderTasks();
}

// 📋 Render Tasks
function renderTasks() {
  const list = document.getElementById("taskList");
  const search = document.getElementById("search").value.toLowerCase();
  list.innerHTML = "";

  let completed = 0;

  tasks.forEach((task, index) => {
    if (!task.text.toLowerCase().includes(search)) return;

    const div = document.createElement("div");
    div.className = "task";
    div.draggable = true;

    if (task.completed) {
      div.classList.add("completed");
      completed++;
    }
  })

  // overdue

```

```

    if (task.dueDate && new Date(task.dueDate) < new Date() &&
!task.completed) {
        div.classList.add("overdue");
    }

    div.innerHTML = `
        <div>
            ${task.text}
            <div class="category">${task.category}</div>
            <div class="due">${task.dueDate ? "Due: " + task.dueDate :
""}</div>
        </div>
        <div class="actions">
            <button onclick="toggle(${index})">✓</button>
            <button onclick="removeTask(${index})">✕</button>
        </div>
    `;

    // Drag events
    div.addEventListener("dragstart", () => {
        div.classList.add("dragging");
    });

    div.addEventListener("dragend", () => {
        div.classList.remove("dragging");
        save();
    });

    list.appendChild(div);
});

// Progress
const percent = tasks.length
    ? Math.round((completed / tasks.length) * 100)
    : 0;

document.getElementById("progressBar").style.width = percent + "%";
document.getElementById("progressText").innerText =
    percent + "% completed";

enableDrag();
}

```

```
// 🔄 Drag Logic
function enableDrag() {
  const list = document.getElementById("taskList");

  list.addEventListener("dragover", e => {
    e.preventDefault();
    const dragging = document.querySelector(".dragging");
    const after = getDragAfterElement(list, e.clientY);

    if (after == null) {
      list.appendChild(dragging);
    } else {
      list.insertBefore(dragging, after);
    }
  });
}

function getDragAfterElement(container, y) {
  const elements =
[...container.querySelectorAll(".task:not(.dragging)")];

  return elements.reduce((closest, child) => {
    const box = child.getBoundingClientRect();
    const offset = y - box.top - box.height / 2;

    if (offset < 0 && offset > closest.offset) {
      return { offset, element: child };
    } else {
      return closest;
    }
  }, { offset: Number.NEGATIVE_INFINITY }).element;
}

// ✔ Toggle
function toggle(i) {
  tasks[i].completed = !tasks[i].completed;
  save();
  renderTasks();
}

// ✖ Delete
function removeTask(i) {
  tasks.splice(i, 1);
}
```

```
    save();
    renderTasks();
}

// 🛎 Reminder checker
setInterval(() => {
    const now = new Date();

    tasks.forEach(task => {
        if (
            task.dueDate &&
            new Date(task.dueDate) < now &&
            !task.completed &&
            !task.alerted
        ) {
            alert("🕒 Task overdue: " + task.text);
            task.alerted = true;
            save();
        }
    });
}, 60000);

renderTasks();
```